

WORD-LEVEL PLAGIARISM CHECKER FOR SHORT DOCUMENTS

Presented By

Sec:10

P.Ruhitha 2520080053

M.Ananya 2520080069

Under the guidance of

K. Chandusha, Asst. Prof., CS&IT



Introduction

Plagiarism refers to the use of another person's content without proper acknowledgment. Manually checking documents for plagiarism can be time-consuming and difficult, especially when multiple documents need to be compared. This project aims to provide an efficient automated solution for detecting copied words and phrases. The system focuses on word-level comparison and demonstrates how data structures and string-matching algorithms can be applied to solve a practical problem. It also emphasizes manual implementation of the algorithms instead of relying on prebuilt data structure libraries.

Abstract

This project presents a Word-Level Plagiarism Checker for detecting copied content in short documents such as essays, assignments, and reports. The system is developed using manually implemented data structures and algorithms without relying on prebuilt library implementations. Documents are tokenized into normalized word sequences and organized using a custom hash table and trie for efficient storage, indexing, and retrieval. Exact plagiarism detection is performed using the Rabin-Karp algorithm with polynomial rolling hash to identify matching words and phrases efficiently. To evaluate the overall similarity between documents, the system performs document similarity analysis using manually implemented Suffix Arrays and Longest Common Prefix (LCP) techniques, enabling the identification of common text segments and similarity scores. The system reports the corresponding source document and similarity percentage for each comparison. The project demonstrates the practical application of advanced string algorithms and custom data structures in building an efficient, reliable, and scalable plagiarism detection system.

Methodology

1. Input Documents – Collect the documents to be compared.
2. Preprocessing – Tokenize and normalize the text into word sequences.
3. Hash Table & Trie – Store and index words for efficient searching.
4. Rabin-Karp Algorithm – Detect matching words and phrases.
5. Suffix Array & LCP – Identify common text segments between documents.
6. Similarity Calculation – Calculate the overall similarity percentage.
7. Result Generation – Display the similarity percentage and source document.

Software Requirements

1

Operating System: Windows

2

Programming Language: Java

3

IDE: Visual Studio Code

4

Input: Text documents (.txt)

5

Version Control: Git

6

Repository Platform: GitHub

Hardware Requirements

- 1 Laptop/Desktop Computer
- 2 Processor: Dual-core processor or higher
- 3 RAM: Minimum 4 GB
- 4 Storage: At least 500 MB free space
- 5 Keyboard and Mouse
- 6 Display Monitor

LITERATURE SURVEY

TURNITIN

- Academic similarity detection system used to compare student submissions with online sources, publications, and previously submitted papers.
- Generates a similarity report highlighting matching text.

MOSS (MEASURE OF SOFTWARE SIMILARITY)

- Developed at Stanford for detecting similarity between programming assignments.
- Compares source code and highlights similar sections

ITHENTICATE

- Similarity-checking system designed mainly for researchers, publishers, and academic content.
- Compares documents against scholarly publications and web content.

A top-down view of a desk with various items: a laptop in the upper center, a cup of coffee on the left, a pen and glasses in the middle, paper clips on the left, and a large green leaf in the bottom left corner. The background is a light, neutral color.

THANK YOU